



EXVUL

SMART CONTRACT **AUDIT REPORT**

AquaFlux Rate Hook Smart Contract

APRIL 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 LiquidityRouter_quoteDeposit Misprices Aq Split on Mixed-Decimal Pools	9
3.2 SwapRouter Omits DualToken-Only Asset Gating and Can Misroute TriToken Settlements	12
3.3 QuoteExactSForAq Can Return an AmountAqSwap That Later Reverts in SwapExactS- ForAq Near Parity	14
3.4 Unnecessary Residual Allowance Granted to Core	18
3.5 SwapRouter Exposes Hook InternalPoolNotInitialized Instead of Enforcing Its Own Pool- NotConfigured Validation	20
3.6 SAFE_MAX Is Applied in Both Raw and Normalized Units	23
3.7 OperationDeadline Is Exposed but Not Enforced	24
3.8 QuoteAddLiquidity Pre-Division Truncation Can Undercharge by One Raw Unit	25
3.9 No Withdrawal Cooldown Enables JIT Liquidity Capture at the Expense of Incumbent LPs	27
4. CONCLUSION	28
5. APPENDIX	29
5.1 Basic Coding Assessment	29
5.1.1 Apply Verification Control	29
5.1.2 Authorization Access Control	29
5.1.3 Forged Transfer Vulnerability	29
5.1.4 Transaction Rollback Attack	30
5.1.5 Transaction Block Stuffing Attack	30
5.1.6 Soft Fail Attack Assessment	30
5.1.7 Hard Fail Attack Assessment	31
5.1.8 Abnormal Memo Assessment	31
5.1.9 Abnormal Resource Consumption	31
5.1.10 Random Number Security	32
5.2 Advanced Code Scrutiny	32
5.2.1 Cryptography Security	32

5.2.2 Account Permission Control	32
5.2.3 Malicious Code Behavior	33
5.2.4 Sensitive Information Disclosure	33
5.2.5 System API	33
6. DISCLAIMER	34
7. REFERENCES	35
8. About Exvul Security	36

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **AquaFlux Rate Hook** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
AquaFlux Rate Hook	20/04/2026 - 27/04/2026	Solidity

Repository

Commit Hash

27e4ad8a32c97fa171a849f57f2e478dea452eb8

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	2
LOW	3
INFO	4



2.3 Key Findings

Severity	Findings Title	Status
MEDIUM	LiquidityRouter_quoteDeposit Misprices Aq Split on Mixed-Decimal Pools	Fixed
MEDIUM	SwapRouter Omits DualToken-Only Asset Gating and Can Misroute TriToken Settlements	Fixed
LOW	QuoteExactSForAq Can Return an AmountAqSwap That Later Reverts in SwapExactSForAq Near Parity	Fixed
LOW	Unnecessary Residual Allowance Granted to Core SwapRouter Exposes Hook	Fixed
LOW	InternalPoolNotInitialized Instead of Enforcing Its Own PoolNotConfigured Validation	Acknowledged
INFO	SAFE_MAX Is Applied in Both Raw and Normalized Units	Acknowledged
INFO	OperationDeadline Is Exposed but Not Enforced	Acknowledged
INFO	QuoteAddLiquidity Pre-Division Truncation Can Undercharge by One Raw Unit	Fixed
INFO	No Withdrawal Cooldown Enables JIT Liquidity Capture at the Expense of Incumbent LPs	Acknowledged

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 LiquidityRouter_quoteDeposit Misprices Aq Split on Mixed-Decimal Pools

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

contracts/src/AquaFluxLiquidityRouter.sol
contracts/src/AquaFluxRateHook.sol
docs/aquaflux-rate-hook.md

DESCRIPTION:

`AquaFluxLiquidityRouter._quoteDeposit` computes the zap split ratio from raw `reserveP` and `reserveUnderlying` units returned by `hook.poolInfo`, without applying the pool's decimal normalization scales. From the hook's design and implementation, explicitly normalize token amounts to WAD so pools with mixed token decimals remain economically comparable.

```
// contracts/src/AquaFluxLiquidityRouter.sol
function _quoteDeposit(bytes32 assetId, PoolKey calldata key, uint256
amountAqIn)
    internal
    view
    returns (DepositQuote memory quote)
{
    AquaFluxRateHook.PoolInfo memory info = hook.poolInfo(key.toId());

    (uint256 netIfAllSplit,) = _previewSplit(assetId, amountAqIn);
    if (netIfAllSplit == 0) revert QuoteNotAvailable();

    uint256 splitNetRate = Math.mulDiv(netIfAllSplit, WAD,
amountAqIn);
    uint256 weightedP = uint256(info.reserveP) * WAD;
    uint256 weightedAq = uint256(info.reserveUnderlying) *
splitNetRate;
    uint256 denominator = weightedP + weightedAq;
    if (denominator == 0) revert QuoteNotAvailable();
```

```
quote.aqToSplit = Math.mulDiv(amountAqIn, weightedP, denominator);  
// ...  
}
```

Inconsistent with hook's normalized liquidity math.

```
// contracts/src/AquaFluxRateHook.sol  
uint256 normalizedReserveP = _toWad(res.reserveP,  
cfg.pTokenScale);  
uint256 normalizedReserveUnderlying =  
_toWad(res.reserveUnderlying, cfg.underlyingTokenScale);  
uint256 sharesFromP = normalizedPDesired * supply /  
normalizedReserveP;  
uint256 sharesFromUnderlying = normalizedUnderlyingDesired *  
supply / normalizedReserveUnderlying;
```

Also inconsistent with project documentation.

```
// docs/aquaflux-rate-hook.md  
The hook normalizes raw token units to 18-decimal WAD math internally, so  
P and RWA tokens may have different decimals.
```

IMPACT:

On a supported mixed-decimal market such as 18-decimal P / 6-decimal aq, `_quoteDeposit` compares `1e18` and `1e6` as if they were directly comparable reserve magnitudes. That makes the decimal gap itself drive the zap split, rather than the pool's normalized economic balance.

RECOMMENDATIONS:

Normalize reserves before computing the split weight.

```
function _quoteDeposit(bytes32 assetId, PoolKey calldata key, uint256  
amountAqIn)  
internal  
view  
returns (DepositQuote memory quote)  
{  
    AquaFluxRateHook.PoolInfo memory info = hook.poolInfo(key.toId());
```

```
        // ...
        uint256 splitNetRate = Math.mulDiv(netIfAllSplit, WAD,
amountAqIn);
-         uint256 weightedP = uint256(info.reserveP) * WAD;
-         uint256 weightedAq = uint256(info.reserveUnderlying) *
splitNetRate;
+         uint256 normalizedReserveP = uint256(info.reserveP) *
info.pTokenScale;
+         uint256 normalizedReserveAq = uint256(info.reserveUnderlying) *
info.underlyingTokenScale;
+         uint256 weightedP = normalizedReserveP * WAD;
+         uint256 weightedAq = normalizedReserveAq * splitNetRate;
        uint256 denominator = weightedP + weightedAq;
        if (denominator == 0) revert QuoteNotAvailable();
    }
```

3.2 SwapRouter Omits DualToken-Only Asset Gating and Can Misroute TriToken Settlements

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`contracts/src/AquaFluxSwapRouter.sol`**DESCRIPTION:**

`_validateAndFetchActive` validates hook address, token addresses, maturity, pause status, and pToken/aqToken pool matching, but never checks `info.assetType == 1`. Inconsistent with `AquaFluxLiquidityRouter._validateMarket`, explicitly rejects non-DualToken assets.

```
// contracts/src/AquaFluxSwapRouter.sol
function _validateAndFetchActive(PoolKey calldata key, bytes32
assetId, uint8 op)
    internal
    view
    returns (MarketTokens memory tokens)
{
    // ...
    IAquaFluxCoreRouter.AssetInfo memory info =
core.getAssetInfo(assetId);
    if (info.pToken == address(0) || info.aqToken == address(0) ||
info.sToken == address(0)) {
        revert AssetTokenMismatch();
    }
    // ...
    tokens = MarketTokens({pToken: info.pToken, aqToken:
info.aqToken, sToken: info.sToken});
}

function _splitAq(RouterCallbackData memory d, uint256 amountAq)
    internal
    returns (uint256 pFromSplit, uint256 sFromSplit)
{
    uint256 pBeforeSplit = IERC20(d.pToken).balanceOf(address(this));
    uint256 sBeforeSplit = IERC20(d.sToken).balanceOf(address(this));
```

```
IERC20(d.aqToken).forceApprove(address(core), amountAq);
core.split(d.assetId, amountAq);

pFromSplit = IERC20(d.pToken).balanceOf(address(this)) -
pBeforeSplit;
sFromSplit = IERC20(d.sToken).balanceOf(address(this)) -
sBeforeSplit;
}
```

IMPACT:

If a TriToken asset is registered against a compatible P/aq pool, swap flows may execute under DualToken assumptions and mishandle settlement outputs. currently referenced Pharos deployment asset queried from Core is `assetType = 1` with `cToken = address(0)`, not triggered by that specific live asset today. But remains unsafe if any future or alternate deployment routes a TriToken asset through it.

RECOMMENDATIONS:

Reject non-DualToken assets in AquaFluxSwapRouter before any quote or swap path proceeds.

```
+   error UnsupportedAssetType();

function _validateAndFetchActive(PoolKey calldata key, bytes32
assetId, uint8 op)
    internal
    view
    returns (MarketTokens memory tokens)
{
    // ...
    IAquaFluxCoreRouter.AssetInfo memory info =
core.getAssetInfo(assetId);
+   if (info.assetType != 1) revert UnsupportedAssetType();
    if (info.pToken == address(0) || info.aqToken == address(0) ||
info.sToken == address(0)) {
        revert AssetTokenMismatch();
    }
}
```

3.3 QuoteExactSForAq Can Return an AmountAqSwap That Later Reverts in SwapExactSForAq Near Parity

SEVERITY:

LOW

STATUS:

Fixed

PATH:

```
contracts/src/AquaFluxSwapRouter.sol  
test/routers/AquaFluxSwapRouter.t.sol
```

DESCRIPTION:

`quoteExactSForAq` uses `_solveAqForSellS` to derive the internal `amountAqSwap`, then returns that value as the recommended parameter for the on-chain solver route. `_sellSFeasible` only checks whether the internal `aq->P` swap returns no more `P` than the user can combine with `S` (`pFromSwap <= amountSIn`). It never checks the execution-side settlement condition that the `aq` returned by `core.combine` must be large enough to repay the internal `aq` debt created by the pool swap (`aqOwed <= netAq`).

```
// contracts/src/AquaFluxSwapRouter.sol  
function _solveAqForSellS(PoolKey calldata key, bytes32 assetId,  
    uint256 amountSIn, bool isPToken0)  
    internal  
    view  
    returns (uint256 aqSwap)  
{  
    uint256 high = amountSIn;  
    while (_sellSFeasible(key, assetId, amountSIn, high, isPToken0)) {  
        if (high > type(uint256).max / 2) revert QuoteNotSolvable();  
        high *= 2;  
    }  
    // ...  
    (, uint256 pFromSwap,) = _quoteSwapSForAq(key, assetId,  
        amountSIn, low, isPToken0);  
    if (pFromSwap > amountSIn || amountSIn - pFromSwap > 1) revert  
        QuoteNotSolvable();  
    aqSwap = low;  
}
```

```
function _sellSFeasible(PoolKey calldata key, bytes32 assetId,
uint256 amountSIn, uint256 amountAqSwap, bool isPToken0)
    internal
    view
    returns (bool)
{
    (, uint256 pFromSwap,) = _quoteSwapSForAq(key, assetId,
amountSIn, amountAqSwap, isPToken0);
    return pFromSwap <= amountSIn;
}

function _executeSellS(RouterCallbackData memory d) internal returns
(uint256 aqOut) {
    // ...
    uint256 netAq = IERC20(d.aqToken).balanceOf(address(this)) -
aqBeforeCombine;
    if (aqOwed > netAq) revert InsufficientAqToSettle();
    // ...
}
```

POC:

```
// test/routers/AquaFluxSwapRouter.t.sol
function testSwapExactSForAq_RevertsNearParity() public {
    // Configure a fresh pool whose P/aq parity stays at 1:1. With
the hook's 5 bps
    // swap fee and zero combine fee in the mock core, buying
~amountSIn P requires
    // strictly more than amountSIn aq while combine only returns
amountSIn aq.
    PoolKey memory parityKey =
_initializePoolWithFee(address(pToken), address(aqToken), 500);
    vm.prank(governance);
    hook.initializePoolConfig(
        parityKey,
        block.timestamp,
        block.timestamp + DURATION,
        WAD,
        WAD,
        WAD,
        WAD,
```

```
        address(pToken),
        INITIAL_A,
        SWAP_FEE_RATE
    );
    _addLiquidityToPool(parityKey, 100_000 ether, 100_000 ether);

    uint256 amountSIn = 1_000 ether;
    (uint256 amountAqSwap, uint256 quotedAqOut, uint256 pFromSwap,) =
        router.quoteExactSForAq(parityKey, assetId, amountSIn);

    // The current solver can still produce a candidate because it
    // only checks the
    // // combine-side P/S match. Near parity, however, the required aq
    // input to buy
    // // ~amountSIn P exceeds the aq returned by combine, so execution
    // reverts.
    assertGt(amountAqSwap, amountSIn, 'solver should require more aq
    than S face value near parity');
    assertLe(amountSIn - pFromSwap, 1, 'solver should still consider
    the route feasible');
    assertEq(quotedAqOut, 0, 'quote should show no residual aq after
    settlement');

    sToken.mint(address(this), amountSIn);
    sToken.approve(address(router), amountSIn);

    vm.expectRevert(AquaFluxSwapRouter.InsufficientAqToSettle.selector);
    router.swapExactSForAq(parityKey, assetId, amountSIn, 0,
    address(this), block.timestamp + 60);
}
```

IMPACT:

In near-parity P/aq markets, Quote is not solvable but `quoteExactSForAq` still returns an `amountAqSwap` and `aqOut` that appear usable to frontends or SDKs, but submitting the corresponding `swapExactSForAq` route still reverts during settlement because the combine output cannot repay the internally borrowed aq. The result is a quote/execute mismatch for the documented automatic `S -> aq` route in that market region.

RECOMMENDATIONS:

Include the aq-debt coverage check in the solver feasibility path. So will return `QuoteNotSolvable` instead of returning an `amountAqSwap` that later reverts when swap.

```
function _sellSFeasible(PoolKey calldata key, bytes32 assetId,
uint256 amountSIn, uint256 amountAqSwap, bool isPToken0)
    internal
    view
    returns (bool)
{
-     (, uint256 pFromSwap,) = _quoteSwapSForAq(key, assetId,
amountSIn, amountAqSwap, isPToken0);
-     return pFromSwap <= amountSIn;
+     (, uint256 pFromSwap,) = _quoteSwapSForAq(key, assetId,
amountSIn, amountAqSwap, isPToken0);
+     if (pFromSwap > amountSIn) return false;
+     uint256 combineAmount = pFromSwap;
+     (uint256 netAq,) = _previewCombine(assetId, combineAmount);
+     return netAq >= amountAqSwap;
}
```

3.4 Unnecessary Residual Allowance Granted to Core

SEVERITY: LOW**STATUS:** Fixed**PATH:**

aquaflux-rate-hook-main/contracts/src/AquaFluxSwapRouter.sol
aquaflux-contracts-main/contracts/libraries/OperationsLib.sol

DESCRIPTION:

The `AquaFluxSwapRouter.sol` contract calls `core.split(d.assetId, amountAq)` to split the AQ tokens. The problem is that the `AquaFluxCore.sol` contract continues to call `OperationsLib.split()`, ultimately executing `burn` to destroy the tokens. No additional authorization is required for this. The same issue also exists in `AquaFluxSwapRouter.sol`'s `_executeSells()` function.

```
// aquaflux-rate-hook-main/contracts/src/AquaFluxSwapRouter.sol
function _splitAq(RouterCallbackData memory d, uint256 amountAq)
    internal
    returns (uint256 pFromSplit, uint256 sFromSplit)
{
    // ...

    IERC20(d.aqToken).forceApprove(address(core), amountAq);
    core.split(d.assetId, amountAq);

    // ...
}
```

```
// aquaflux-contracts-main/contracts/libraries/OperationsLib.sol
function _split(
    mapping(bytes32 => IAquaFluxCore.AssetInfo) storage assets,
    mapping(bytes32 => uint256) storage assetUnderlyingBalances,
    mapping(bytes32 => mapping(IAquaFluxCore.OperationType =>
uint16)) storage assetFeeRates,
    mapping(bytes32 => uint256) storage assetFeesTotal,
    mapping(bytes32 => mapping(string => uint256)) storage
assetFeesByOperation,
```

```
mapping(bytes32 => uint256) storage assetFeesWithdrawable,
ITokenFactory factory,
bytes32 assetId,
uint256 amount,
address user
) internal returns (uint256 netAmount, uint256 feeAmount) {
    // ...

    // Burn AqTokens from user
    IBaseToken(asset.aqToken).burn(user, amount);

    // ...

}
```

IMPACT:

This issue does not directly cause `split()`/`combine()` to fail under the current implementation, nor does it result in immediate financial loss, but it exposes unnecessary token control permissions to the Core.

RECOMMENDATIONS:

Directly remove the `forceApprove` operation or clear the allowance after executing `split()`/`combine()`.

3.5 SwapRouter Exposes Hook InternalPoolNotInitialized Instead of Enforcing Its Own PoolNotConfigured Validation

SEVERITY:

LOW

STATUS:

Acknowledged

PATH:

```
contracts/src/AquaFluxSwapRouter.sol
contracts/src/AquaFluxRateHook.sol
```

DESCRIPTION:

[AquaFluxSwapRouter](#) defines a router level [PoolNotConfigured](#) error and integration docs say the router validates that the hook pool is configured, but [_validateAndFetchActive](#) never performs that check.

```
// contracts/src/AquaFluxSwapRouter.sol
error PoolNotConfigured();

function _validateAndFetchActive(PoolKey calldata key, bytes32
assetId, uint8 op)
    internal
    view
    returns (MarketTokens memory tokens)
{
    if (address(key.hooks) != address(hook)) revert InvalidPoolKey();
    if (Currency.unwrap(key.currency0) == address(0) ||
Currency.unwrap(key.currency1) == address(0)) {
        revert InvalidPoolKey();
    }

    IAquaFluxCoreRouter.AssetInfo memory info =
core.getAssetInfo(assetId);
    if (info.pToken == address(0) || info.aqToken == address(0) ||
info.sToken == address(0)) {
        revert AssetTokenMismatch();
    }
    if (block.timestamp >= info.maturity) revert AssetExpired();

    if ((info.pausedOperations & op) != 0) {
```

```
        if (op == PAUSE_SPLIT) revert SplitPaused();
        if (op == PAUSE_COMBINE) revert CombinePaused();
    }

    address c0 = Currency.unwrap(key.currency0);
    address c1 = Currency.unwrap(key.currency1);
    bool pIsC0 = (c0 == info.pToken && c1 == info.aqToken);
    bool pIsC1 = (c1 == info.pToken && c0 == info.aqToken);
    if (!pIsC0 && !pIsC1) revert AssetTokenMismatch();

    tokens = MarketTokens({pToken: info.pToken, aqToken:
info.aqToken, sToken: info.sToken});
    }

    function quoteExactAqForS(PoolKey calldata key, bytes32 assetId,
uint256 amountAqIn)
        external
        view
        returns (uint256 amountPSwap, uint256 sOut, uint256 aqFromSwap,
uint256 splitFee)
    {
        if (amountAqIn == 0) revert ZeroAmount();
        MarketTokens memory tokens = _validateAndFetchActive(key,
assetId, PAUSE_SPLIT);
        bool isPToken0 = tokens.pToken == Currency.unwrap(key.currency0);

        amountPSwap = _solvePForBuyS(key, assetId, amountAqIn, isPToken0);
        (sOut, aqFromSwap, splitFee) = _quoteSwapAqForS(key, assetId,
amountAqIn, amountPSwap, isPToken0);
    }
}
```

```
// contracts/src/AquaFluxRateHook.sol
function _loadConfiguredPool(PoolKey memory key) internal view
returns (PoolId id, PoolState storage pool) {
    _validatePoolKey(key);
    id = key.toId();
    pool = pools[id];
    if (pool.config.pInitialValue == 0) revert PoolNotInitialized();
}
```

IMPACT:

Leaves `PoolNotConfigured` effectively dead for this scenario, and forces frontends/SDKs to understand hook internal revert selectors even though the router claims to validate configured pools itself.

RECOMMENDATIONS:

Add an explicit hook pool configuration check.

```
function _validateAndFetchActive(PoolKey calldata key, bytes32
assetId, uint8 op)
    internal
    view
    returns (MarketTokens memory tokens)
{
    // ...

+     AquaFluxRateHook.PoolInfo memory pool =
hook.poolInfo(key.toId());
+     if (!pool.configured || pool.lpToken == address(0)) revert
PoolNotConfigured();

    address c0 = Currency.unwrap(key.currency0);
    address c1 = Currency.unwrap(key.currency1);
    bool pIsC0 = (c0 == info.pToken && c1 == info.aqToken);
    bool pIsC1 = (c1 == info.pToken && c0 == info.aqToken);
```

3.6 SAFE_MAX Is Applied in Both Raw and Normalized Units

SEVERITY:

INFO

STATUS:

Acknowledged

PATH:`contracts/src/AquaFluxRateHook.sol`**DESCRIPTION:**

AquaFluxRateHook applies `SAFE_MAX` checks in two different unit systems: raw token units and normalized 18-decimal units. Because normalized values are computed as `rawAmount * scale`, tokens with decimals < 18 are subject to a stricter effective raw-token upper bound than the raw `SAFE_MAX` constant suggests. It is a conservative boundary choice that can be confusing to read and reason about. In `_quoteAddLiquidity()`, the code checks both:

```
if (amountPDesired > SAFE_MAX || amountUnderlyingDesired > SAFE_MAX)
    revert ReserveOverflow();
and:
uint256 normalizedPDesired = _toWad(amountPDesired, cfg.pTokenScale);
uint256 normalizedUnderlyingDesired = _toWad(amountUnderlyingDesired,
    cfg.underlyingTokenScale);
if (normalizedPDesired > SAFE_MAX || normalizedUnderlyingDesired >
    SAFE_MAX) revert ReserveOverflow();
```

IMPACT:

For low-decimal tokens, the normalized check becomes the tighter one. So although the raw check appears to allow values up to `1e26`, the normalized check will reject much smaller raw amounts. It makes the practical bound stricter than the raw constant name alone suggests.

RECOMMENDATIONS:

Replace the shared `SAFE_MAX` constant with distinct constants for raw and normalized domains.

3.7 OperationDeadline Is Exposed but Not Enforced

SEVERITY: INFO

STATUS: Acknowledged

PATH:

contracts/src/interfaces/IAquaFluxCoreLiquidity.sol

DESCRIPTION:

The struct `AssetInfo` function in the `IAquaFluxCoreLiquidity` interface has been updated with `operationDeadline`, but this is not being validated in the currently audited contracts.

```
interface IAquaFluxCoreLiquidity {
    struct AssetInfo {
        address issuer;
        address underlying;
        uint256 maturity;
        uint256 operationDeadline;
        // ...
    }
    // ...
}
```

IMPACT:

The `operationDeadline` field is exposed in the interface to the core layer, but the router does not use it. If this field is intended to limit operation time, omitting it may currently lead to unexpected behavior.

RECOMMENDATIONS:

No recommendation was provided in the issues list.

3.8 QuoteAddLiquidity Pre-Division Truncation Can Undercharge by One Raw Unit

SEVERITY:

INFO

STATUS:

Fixed

PATH:`contracts/src/AquaFluxRateHook.sol`**DESCRIPTION:**

`_quoteAddLiquidity()` computes `amountP` and `amountUnderlying` by dividing normalized space before rounding up back to raw token units:

```
amountP = _fromWadUp(shares * normalizedReserveP / supply,
    cfg.pTokenScale);
amountUnderlying = _fromWadUp(shares * normalizedReserveUnderlying /
    supply, cfg.underlyingTokenScale);
```

This is not equivalent to rounding up the raw-token result of `shares * reserve / supply`. The intermediate division truncates first, so the final `_fromWadUp(...)` can no longer recover the lost fraction. As a result, the quoted raw token amount can be lower than the mathematically correct ceil-rounded amount.

IMPACT:

A liquidity add can undercharge up to 1 raw unit per token side. In edge cases, an amount that should round up to 1 raw unit can become 0, causing `ZeroShares()` and rejecting an otherwise valid small deposit. The issue only affects tokens with decimals < 18 (`scale > 1`).

RECOMMENDATIONS:

Round up in raw token space directly:

```
function _quoteAddLiquidity(PoolId id, Config storage cfg, uint256
    amountPDesired, uint256 amountUnderlyingDesired)
    internal
    view
```

```
        returns (uint256 shares, uint256 amountP, uint256
amountUnderlying)
{
    // ...
    amountP = _fromWadUp(shares * normalizedReserveP / supply,
cfg.pTokenScale);
    amountUnderlying = _fromWadUp(shares * normalizedReserveUnderlying /
supply, cfg.underlyingTokenScale);
    amountP = Math.ceilDiv(shares * res.reserveP, supply);
    amountUnderlying = Math.ceilDiv(shares * res.reserveUnderlying,
supply);
    // ...
}
```

3.9 No Withdrawal Cooldown Enables JIT Liquidity Capture at the Expense of Incumbent LPs

SEVERITY:

INFO

STATUS:

Acknowledged

PATH:`contracts/src/AquaFluxRateHook.sol`**DESCRIPTION:**

AquaFluxRateHook allows liquidity to be added immediately before a swap and removed immediately after it, while swap fees and rebalancing gains remain embedded in reserves and are shared pro rata by the LP set present at execution. The severity of this issue depends on protocol design intent: if instant, permissionless LP entry and exit is an explicit design choice, then this behavior is a consequence of that choice rather than a vulnerability.

IMPACT:

Incumbent LPs bear the inventory risk during the imbalance phase but cannot retain the full economic benefit of the later rebalancing swap, because a short-term LP can enter only for the profitable recovery window and exit immediately afterward to capture part of the embedded fee and rebalancing gain.

RECOMMENDATIONS:

Add a minimum holding period or withdrawal cooldown for newly added liquidity.

4. CONCLUSION

In this audit, we thoroughly analyzed **AquaFlux Rate Hook** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec